ORIGINAL CONFERENCE · GETTING STARTED

The project, and your first task

What the project is, then every step of a task: the injection, the prompt, Unique Ground Truth, Oracle Events and the rubric.



~45

minutes · 9 parts

Current guidelines as of 23 October 2026 · Screens from the live task

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-11-04T12:24:12.070Z

The project, and your first task in the order you'll do it

0	1	2	3	4	5	6	7	8
The project	Assignment	Tools	Seed the universe	The prompt	Unique Ground Truth	Oracle Events	The rubric	Review and submit
3 min	4 min	3 min	5 min	8 min	5 min	4 min	9.5 min	2 min

In the task: injection › prompt › Opus 5.5 pre-check › six baseline runs › Oracle Events › rubric › verifier and difficulty check

About 45 minutes: a minute and a half of welcome, then 43.5 across the nine parts.

What this project is, and how it's judged

A realistic work request for an AI agent inside HarmonyGames, a founder-led mobile game studio. You write it as one of the studio's people; Opus 5.5 runs it six times; your rubric grades every run.

1 Seed
Optional SQL that adds the evidence

2 Prompt
The request, in your persona's voice

3 Oracle Events
The steps a correct agent takes

4 Rubric
Weighted criteria that grade it

5 Baseline
Six Opus 5.5 runs, graded

1 · THE REVIEWS

- /injection, /prompt, /hardness, /oe, /rubric
- At least NON-FAIL; a FAIL blocks

2 · THE BASELINE

- Average under your mode's cap
- pass@1 at most 67% (pass@k)
- 2+ failure patterns, enough tool calls

3 · QC REVIEW

- Five dimensions, each 1 to 5
- The lowest dimension is the score
- Use QC Companinion from instructions/guidelines set

Hard is fine. Unfair is the problem. The agent should fail for a real weakness, caught by a row a careful agent would pass.

HarmonyGames, frozen at a moment in time

The flagship

A domino-matcher, the studio's established revenue source

The newer title

A zombie match-3, still finding its audience

The latest launch

A fighting game, the most recently launched

19 personas across design, engineering and the executive team. 13 services.

OCTOBER CUT · 75% OF TASKS

October 1, 2025

00:00 UTC

A Wednesday

FEBRUARY CUT · 25% OF TASKS

February 14, 2026

00:00 UTC

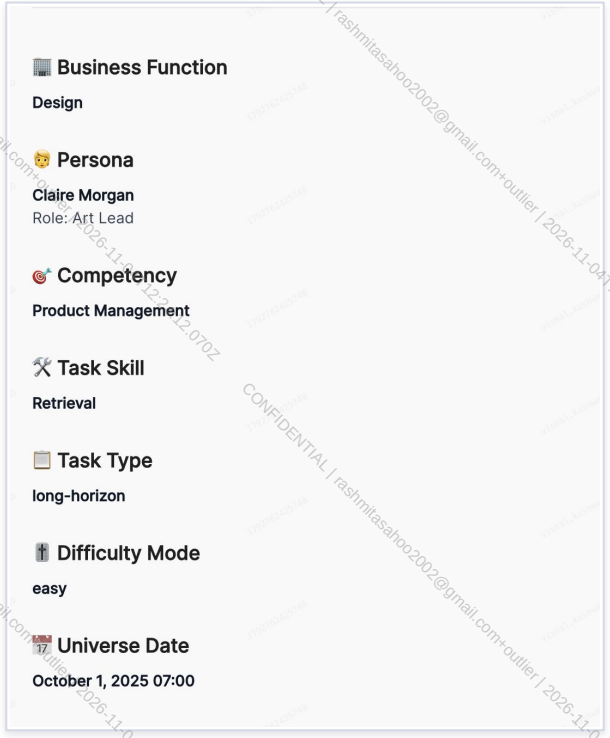
A Saturday

- Before the clock is the past. Nothing after it has happened.
- Write dates absolutely: "february 15", never "yesterday".
- Today in a prompt is the clock's date.
- People read Pacific; SQL timestamps are UTC (+00).
- When the day differs, write both: "Nov 16 PT (Nov 17 UTC)".

Your cut is assigned. Always check your universe date; don't assume it from your last task.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-11-04T12:24:12.070Z

Your assignment is fixed before you start



Your values, at the top of the task

Axis	What it sets	If it doesn't fit
Business function	The part of the company	Non-fail if ambiguous
Persona	Who asks, and what the agent reads	Fail if implausible
Role	The persona's job title	Fail if mismatched
Competency	The type of work (next slide)	Non-fail if ambiguous
Task skill	What the agent does (next slide)	Non-fail if weak
Task type	Normal 50 to 100 tool calls; Long-Horizon 100 to 500+	Fail if under (Wrong Horizon)
Universe date	Feb 14, 2026, 00:00 UTC, or Oct 1, 2025	Fail if built on the other date
Difficulty mode	Average at most 90% Easy, 85% Medium, 70% Hard; pass@1 at most 67% (pass@k)	Fail if over its ceiling

Today in a prompt is the clock's date. PT for people, UTC for SQL.

Competency and task skill: what kind of work it is

COMPETENCY

The type of work

It doesn't have to be the obvious match for your persona. Let it decide what kind of work the task is.

- Game Engineer + Product Management: a product-strategy or coordination task, not a coding one
- CTO + Finance: budgeting or forecasting

PRODUCT MANAGEMENT, FOR EXAMPLE

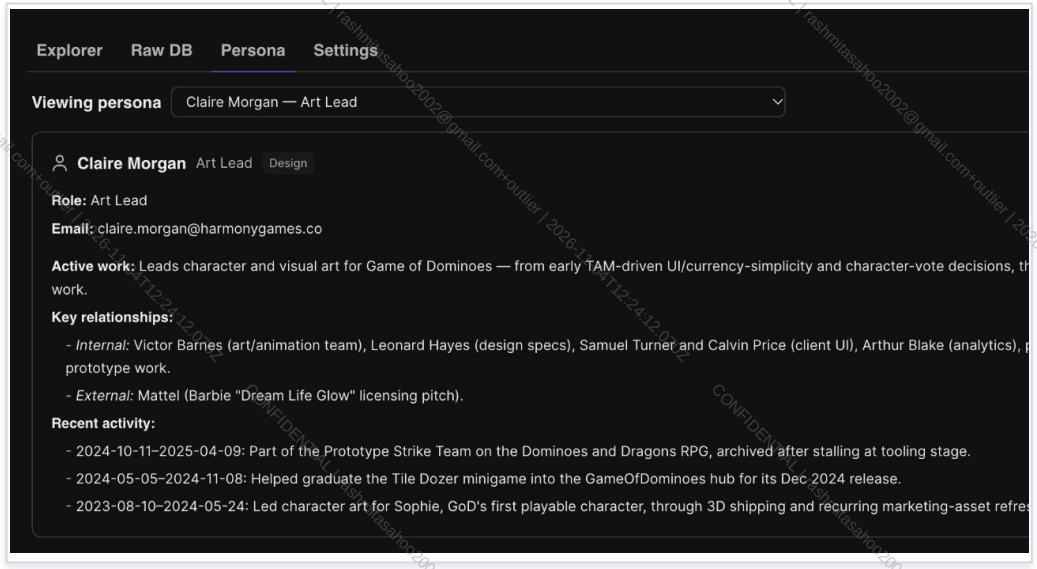
Product strategy, design and cross-functional communication. Outputs: PRDs, spreadsheets, strategy decks, emails.

TASK SKILL: WHAT THE AGENT HAS TO DO

Retrieval	Find and return specific information across services *Pull a budget figure from a sheet and check it against Slack*
Analysis and synthesis	Combine sources, then compute, reconcile or diagnose *Reconcile conflicting revenue numbers across a sheet and a slide deck*
Planning and decision support	Recommend a course of action or a plan, with tradeoffs *Propose a sprint plan from the Linear backlog and team calendars*
Authoring	Produce a deliverable: email, doc, deck, sheet or report *Write a post-mortem from incident data in Slack, Linear and GitHub*
Actions	Create or update records, changing system state *Create Linear tickets for every open bug in a Slack thread*

Stack for strength. Your assigned skill must be clearly present; the best tasks add others, like Retrieval, then Analysis, then Actions.

Your persona: who is asking, and what the agent can see



The Persona Summary tab

USE THE PERSONA YOU WERE ASSIGNED

Choosing a different one fails the task.

WHAT IT CAN READ

SCOPED Slack, Gmail, Calendar, Drive, Docs, Sheets, Slides: only this persona's view, even by ID

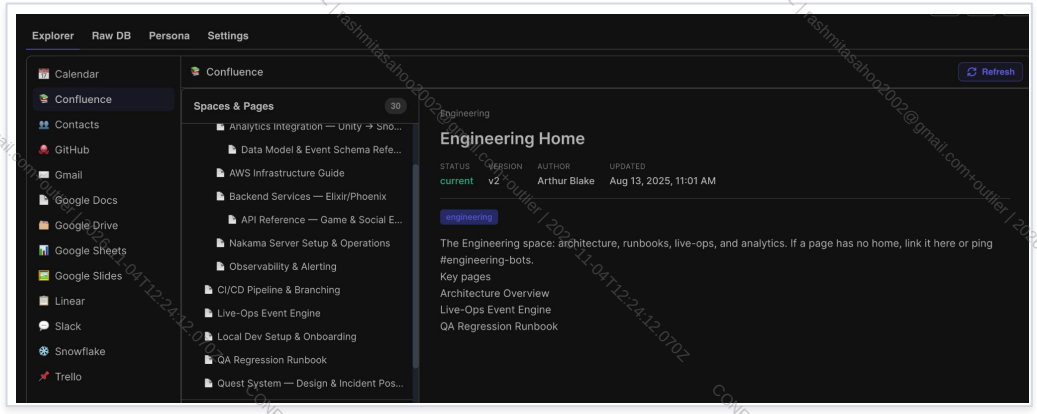
SHARED GitHub, Linear, Trello, Contacts: every persona sees them

VERIFY FIRST Confluence and Snowflake: check from the persona's view

WHAT IT CAN WRITE

Check Agent tool access. Writes come from the tool catalog, not from read visibility.

The Universe Explorer: four tabs, one environment



Settings: expiry, snapshot, load



Settings: redeploy

Persona Summary who you're writing as

Universe browse the company data

Database run SQL: this is where your injection loads

Settings expiry, snapshot, load and redeploy

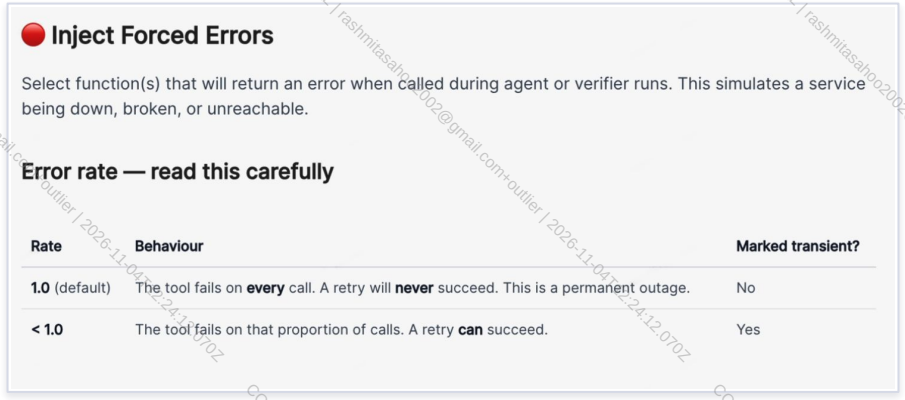
ABOUT 24 HOURS

Then the environment expires. Snapshot your universe, and keep your SQL.

LOADING WIPES CHANGES

Loading a new universe wipes any changes you made.

Optional: failing tools, slow search and reference files



Inject Forced Errors

Select function(s) that will return an error when called during agent or verifier runs. This simulates a service being down, broken, or unreachable.

Error rate — read this carefully

Rate	Behaviour	Marked transient?
1.0 (default)	The tool fails on **every** call. A retry will **never** succeed. This is a permanent outage.	No
< 1.0	The tool fails on that proportion of calls. A retry **can** succeed.	Yes

The error-rate setting in the task

Error rate 1.0 fails every call: tests a workaround or escalation

Below 1.0 fails some calls: tests retries

Async wait search becomes submit, check, get: 30 to 35 seconds a call

Still solvable.

Name the alternative path, confirm it exists, and put it in your Oracle Events.

Async affects search only.

If the prompt names the exact artifact, the agent fetches it and never waits.

Prefer unscoped services for errors.

A forced error on a scoped tool looks exactly like an ACL denial.

REFERENCE FILES, OPTIONAL

Upload a folder only if the agent must read an attachment. Upload the same files again on the next step: that copy is the record.

Seed the symptom, never the conclusion

Rules for valid, consistent injection 🔴

Injected data has to hold up under investigation. Before you rely on anything you add, make sure it's:

1. **On the clock.** No date/time is ever after your assigned current date — **October 1, 2025 07:00**. No future events written as already done; business messages on weekdays at plausible hours; a record's created time is never after its updated time.
2. **Consistent across services.** If you add a Slack thread about a pull request, that PR must actually exist in GitHub. Names, IDs, dates, and facts must line up everywhere they appear — edit one side, fix the other.
3. **Coherent with the universe.** New data fits HarmonyGames' existing storylines, people, games, and relationships, and doesn't contradict what's already there (e.g., don't set a doc to $65 while a linked sheet still says $35).
4. **Structurally valid.** Records fit the real table schema (click here) and follow the universe's existing ID formats (e.g., `ENG-####`, `ZOM-###`). No invented fields or malformed IDs.
5. **Reachable & findable.** Evidence the agent needs is readable by your assigned persona (mind the scoped services) and discoverable by a realistic search — no references pointing to records that don't exist, and nothing buried where search can't reach it.
6. **Natural.** Match each service's real voice, length, and formatting. Avoid AI-tells — generic phrasing, repeated boilerplate, implausibly tidy or complete text.
7. **Not pre-solving.** Seed the *situation* and the *evidence*, not the conclusion. The agent should still have to connect the dots.
8. **State matches activity.** A record's status must agree with what happened to it: no ticket marked Closed whose PR never merged, no task Done with an open blocker, no invoice Paid with no payment. A single cosmetic mismatch that doesn't affect solvability is a minor issue, but still fix it.

You'll record whether you injected data — and attach the SQL + changelog — on the next step.

🔑 The move behind all three: Data seeding

Every lever above works the same way — **you put the complexity in the universe, not in the prompt.** *Data seeding* is how you do that: add or edit universe data (a Slack thread, a ticket, a doc, a conflicting spreadsheet) so the difficulty lives in what the agent has to discover.

A common use is keeping the prompt short and natural:

❌ **Don't** cram it into the prompt: *"Hi, can you help me do... [long, detailed instructions]"*

✅ **Do** seed it: put the instructions in a Slack channel, then prompt: *"Can you check #channel-name and follow the instructions there?"*

*"⚠️ Call this **data seeding / injection** — not "SQL injection," which means a security vulnerability.*

Seed the context, don't type it into the prompt

PRE-SOLVED

A seeded record that states the root cause. The agent has nothing left to find.

SEEDED WELL

The symptom and the evidence, spread across services. The agent connects the dots.

Create the injection: describe it, the web eval writes the SQL

ILLUSTRATION

Write the injection SQL for three September UA bills in Frederick's Gmail: payment summaries dated Sep 16, 18 and 20, 2025, for $103.20, $33.80 and $35.10.

Here's the SQL. Paste it into Database, then Query:

```
BEGIN;
INSERT INTO gmail.gmail_threads
  (id, user_email, snippet, history_id)
VALUES ('frederick.stone@harmonygames.co/18436...',
  'frederick.stone@harmonygames.co',
  'Payment summary. Amount billed: $103.20 ...', NULL);
COMMIT;
```

CHECK IT BEFORE YOU RUN IT

Real IDs. Copy IDs that already exist; new rows get unique IDs.

UTC, on the clock. Timestamps end in +00 and sit at or before the clock.

Readable. Your persona can open every record the task depends on.

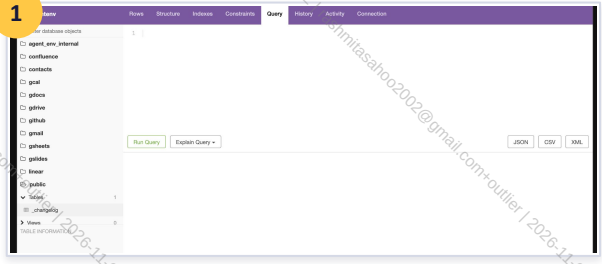
Consistent. Names, IDs, dates and figures match across services.

Not pre-solved. Seed the symptom, not the conclusion.

Snowflake can't be injected. It's a separate analytics database; the injection file can't change it.

Run the injection, then prove it with /injection

1

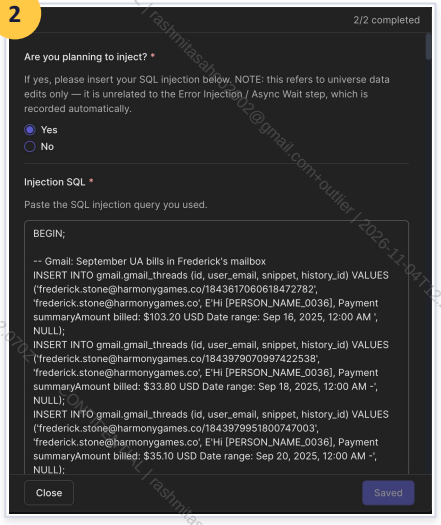


Database, Query: paste, then Run Query

DUPLICATE KEY ERROR?

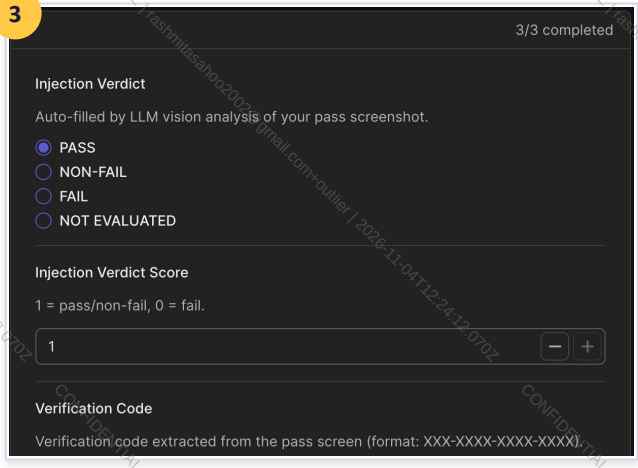
That ID already exists. Delete it first, then insert, so the file can run twice.

2



Paste the same SQL into the Injection SQL field

3



Run /injection, then enter the verdict, score and verification code

At least NON-FAIL to continue. A FAIL blocks the task: fix the SQL and rerun /injection. No proof upload needed.

What a good prompt is, and the shape that works

Who asks

Your persona, in their own voice, about work they'd really be pulled into

What's wrong

A visible symptom, said the way a colleague would say it

Investigate

Work out what's actually true, across services

Act

Writes in two or more services, with the destination and any deadline named

STATES THE CAUSE: PRE-SOLVED

"The CPI jump is the Adjoe incentivized test. Write that up for Robert."

NAMES THE SYMPTOM

"CPI jumped last week and I don't know which test did it. Tell Robert what the records support, and say so if they don't settle it."

Rule zero: seed the context, don't type it. Vague about where the answer is, clear about what to produce and where it goes.

A real request, in your own words



ALWAYS

- Require real tool use
- Span at least two or three services
- Sound like a real person typing
- Write dates absolutely
- Ask for investigation and action
- Name where the output goes, when the person would know it

NEVER

- Name a tool, a parameter or an MCP server
- State the root cause (that pre-solves it)
- Include internal IDs a person wouldn't know
- Announce an injected failure or wait
- Format it like a document, a spec or a step list
- Pose a riddle, hint at the trap, or add an answer-key clause
- Reuse the same setup in every task: "actually", "don't trust the summary"

Clean spelling is fine. Staged typos, broken spacing and emoji the workspace never uses read as contrived.

Looks wrong, but passes

✓ A deadline after the clock

Fine when the event is really upcoming: "before the accounts are archived on february 15".

✓ Casual, lowercase typing

That's how people write in chat. Clean, correct spelling is fine too.

✓ Naming a service or a file

"check slack", "the design doc". Only an actual tool name fails.

✓ Not saying how many

"tickets for whatever someone needs to pick up": the evidence decides.

✓ Asking for more than one write

A record and tickets: a task needs several meaningful writes.

✓ An exact, readable title

GEOGAMES_SPEC.docx is fine. A 30-character machine file name isn't.

✓ A character who is wrong

A seeded message can be false on purpose. That's the trap, not a truthfulness error.

✓ A cause nobody mentions

Good difficulty, as long as nothing in the prompt hints at it.

Test each phrase: would a real colleague write it, and is it true? Then it isn't the error.

Where difficulty comes from

The one rule: difficulty comes from the environment, never from a vague prompt. Scattered and stale is hard; unclear is a FAIL.

SIX CHARACTERISTICS, ALL OPTIONAL, ALL STACKABLE

Ambiguity

SEEDING

The prompt under-specifies; the agent fills the gaps from the environment

Dynamic change

SEEDING

An early finding is overturned by one the agent reaches later

Multi-source

SEEDING

Three or more sources, with distractors and a disagreement

Async tool calls

CONFIGURATION

Submit, then wait: the agent parks it and resumes

Error recovery

CONFIGURATION

Forced failures: the agent retries, works around or escalates

Access control

PERSONA AND SEEDING

The agent works within what its persona may read

HOW AGENTS SEARCH

Naming the artifact kills discovery. An exact title or ID turns the search into one fetch.

Agents stop early. A message that sounds final ends the investigation: a fair trap.

Agents skip a slow path. If you set an async wait, check the run actually took it.

IMPLICIT REQUIREMENTS

Encouraged when a competent assistant would infer them, like checking the data before filing a bug. Never a hidden gotcha.

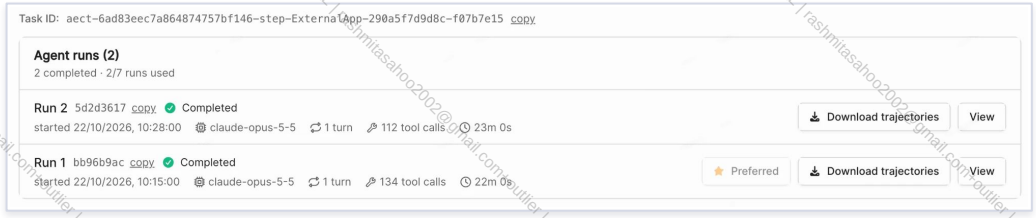
Every rubric row traces back to the prompt. A seeded requirement the prompt gives no reason to find becomes an unauthorized criterion.

How prompts fail, and how they're graded

Area	FAIL	NON-FAIL
Ground truth	Two fair readings end in different states, and the rubric accepts only one	No middle band: 5, or 1 to 2
Clarity	The intent can't be recovered, or two readings lead to different writes	The wording varies, but the action doesn't
Naturalness	Contrived: spec language, a trap-hint riddle, an answer-key clause, staged typos	One tell, like a full internal ID
Tools	An actual tool name: "run linear_create_issue for each item"	Naming the server, or "the X tool"
Facts	A wrong channel, ID, amount or date; a missing record; two loose slips	One loose slip, like a wrong role title
Depth	One service answers it, a second changes nothing, or the cause is handed over	None: each of these fails
Persona	It couldn't plausibly come from your assigned persona	Plausible, but a stretch for the role
Extra asks	A bolt-on: "...and book a dentist for Thursday"	A secondary ask that can't be done

Facts: one tight error fails. A wrong channel, ID, amount or date fails alone; loose slips fail in pairs. Characters may be wrong; the prompt can't.

Test it once, then start the six-run baseline



Pre-check: one Opus 5.5 run, then /hardness



Baseline: six runs that finish in the background

PROCEED

It failed on substance, missed a cross-service link, or stopped at a message that only sounded final.

GO BACK

It solved the task, only missed something cosmetic, or found everything in one or two services.

DON'T SIT AND WATCH

Start the six runs, click Next, and write your Oracle Events and rubric while they run.

Don't tune to the baseline. Revising the prompt until it passes destroys the only difficulty signal the task has.

Unique Ground Truth: one destination, whatever the route

1 Decision	**2 Writes**	**3 Destination**	**4 Contents**
What is true, or what gets decided	Create, update, close, send, or leave it	Which doc, ticket, channel or sheet	The facts that must appear

PASSES: THE UNIVERSE SETTLES IT

Discovery ambiguity: no source is named, but every run ends in one state

A newer record wins: a later, dated update replaces the old one

A documented rule: written where the agent will look

Same facts, other words: two phrasings of the same end state

FAILS: THE RUBRIC DOESN'T HANDLE IT

The rubric picks one reading: two fair readings, only one passes

A disjunctive criterion: "a doc or a Linear issue" names both

An answer-key clause: "if two places differ, put both"

Grading only your reading: it fails an agent that read the prompt accurately

Scored 5, or 1 to 2. Passes when the rubric accepts every fair reading. A later rubric row can't make an open prompt unique.

The two-readings test, and the five checks

Check	Reading A	Reading B
Decision	What is true, or what gets decided	The other defensible version
Writes	Create, update, close, send, or leave it	The other set of actions
Destination	Which doc, ticket, channel or sheet	The other home
Contents	The facts that must appear	The facts that differ

FIVE PLACES THAT USUALLY SPLIT

Which items, which number, which source wins, what counts as the event, which action.

WHAT A REVIEWER USES TO PICK A SIDE

1. **Newer source**
   A later record of the same fact can replace an earlier one. A missing one can't.
2. **Authority**
   Did the prompt say to follow a particular source? Quote that sentence.
3. **Who said it**
   Is it the person or file the prompt named? Someone else's words aren't the same fact.
4. **Scope**
   Does the record settle the whole task, or only one part?
5. **Same result**
   Same decision, writes and contents? The same destination isn't enough.

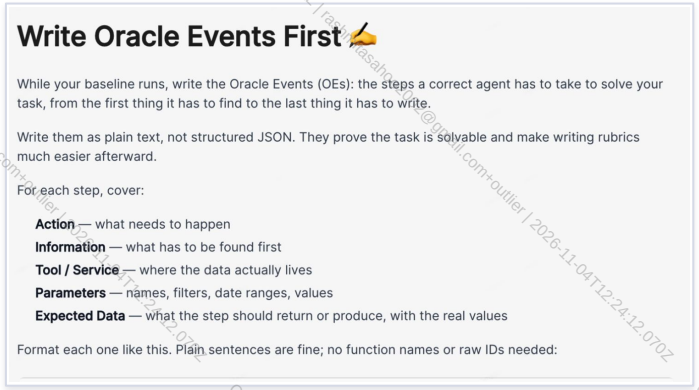
Scored 5, or 1 to 2. If no check kills the second reading, it stays live. Fix the prompt, or accept both in the rubric.

Patterns that leave two readings, and the fix

Pattern	Bad: two readings stay alive	Good: one reading
A scope phrase that stretches	"Clear everything still sitting on my plate before we shut down."	"Clear the items Arthur listed in his message this morning. Leave anything he did not list alone."
Two homes for one deliverable	"Write up the open items in one tracker so we do not lose them."	"Put the open items on the existing Linear project. Do not make a Google doc."
A verb that allows two actions	"Go through the leftover reviews and wrap them up or flag them."	"If a review already has an owner, comment on it and leave it open. If nobody owns it, close it."
Two numbers, and no rule	"The build has to come in under 90 MB. Tell me what we still need to cut."	"Use the latest size measurement of the build we are actually shipping."
Two sources, nothing ranks them	"Tell me which screenshots are actually live. Go by what really happened."	"Use the store log. If a later message only remembers a different set, the log wins."

The good line picks one side without naming the answer: the agent still has to find the items, the project or the log.

Oracle Events: what they're for, and one good event



Write Oracle Events First ✍️

While your baseline runs, write the Oracle Events (OEs): the steps a correct agent has to take to solve your task, from the first thing it has to find to the last thing it has to write.

Write them as plain text, not structured JSON. They prove the task is solvable and make writing rubrics much easier afterward.

For each step, cover:

- **Action** — what needs to happen
- **Information** — what has to be found first
- **Tool / Service** — where the data actually lives
- **Parameters** — names, filters, date ranges, values
- **Expected Data** — what the step should return or produce, with the real values

Format each one like this. Plain sentences are fine; no function names or raw IDs needed:

Field	Example, from the Example Task
Action	Read the February 11, 2026 playable thread to its end, not to 21:08 UTC, where it sounds finished
Information	Whether the playable shipped, and with which links
Tool / Service	Slack: the #prototype channel history
Parameters	Channel #prototype, 2026-02-11
Expected data	Unity rejects it at 21:03 for a missing store URL; workaround 21:10; "it works" 21:22; paid campaign on 21:37; iOS-only confirmed 21:54. All UTC.

Prove it's solvable. They show a correct path exists.

Drive the rubric. Every action becomes an Outcome row.

Typed by hand. Plain text; the field blocks pasting.

Not a golden run. They don't count toward the tool-call minimum.

/oe passes or blocks. It has no NON-FAIL: fix anything it flags. Write each as: OE <n> <what, and where> Expected: <the actual values>.

Oracle Event mistakes, and how they're scored

Complete: the full path, its dependencies, every write, and the recovery or wait path as its own event.
Accurate: the right service, exact values, times with a zone. Check every ID in the explorer, then read the set as the agent would.

Mistake	Weak	Better	QC severity
Wrong service (inaccurate)	"Look up the refund total in Slack"	Finance sheet, card refunds for Feb 25-27. Expected: 61 refunds, $4,120	Non-fail
Fact from nowhere (incomplete)	"OE 5: Raise a ticket for the Android link"	First the event that finds the link, then the one that acts on it	Non-fail
Missing write (incomplete)	The events stop after the reads	Each write the prompt asks for gets its own event	Non-fail
Contradicted value (inaccurate)	"The test started Feb 12"	The date the record shows: "Started Feb 10"	Non-fail
A non-action	"The agent doesn't stop at the first message"	"Read the thread to its end. Expected: ..."	Moderate
Final-reply content	"The reply explains that Meta was the only paid network"	Turn it into the discovery step that found it	Moderate
Rounded figure (minor)	"about $4k"	"$4,120 across 61 refunds"	Non-fail

/oe passes or blocks. It has no NON-FAIL, so fix anything it flags. The severities are how QC scores the defect afterwards.

From Oracle Events to criteria

```
OE 4  Aggregate paid spend by network. Expected: 152
rows, channel = meta on all of them, 4510.33 USD.
```

› [wt 3] Agent's record states total paid spend of $4,510.33 across the campaign window.

```
OE 9  Notify the owner, then schedule the review.
```

› [wt 3] Agent notifies the owner of the discrepancy. [wt 3] Agent schedules the review. [wt 1, Process] Agent notifies the owner before scheduling the review.

FROM THE ORACLE EVENTS GUIDELINE: EVENT ON THE LEFT, CRITERIA ON THE RIGHT

1 Every action, an Outcome row

Plus a content row for every key fact the user asked for.

2 Look for gaps

Requirements no Outcome can verify: almost always ordering.

3 Process only if all three hold

Required by every valid path · no stricter Outcome works · a behaviour, not a trace.

Write every row yourself. No LLM. Prefill is off and the Criteria Builder opens empty on purpose; /rubric only audits what you wrote.

Anatomy of a criterion, and the rules

Agent's write-up	identifies	the $4,180 variance between the finance sheet and the publisher statement
WHO, AND THE SURFACE	ACTIVE VERB	THE EXACT VALUE

Outcome first. Process only for ordering the final state can't prove.

Agent + active verb. Never passive, never a tool name.

Affirmative. "Leaves ENG-123 open", not "does not close ENG-123", even when the prompt forbids it.

Self-contained, atomic. The real value, one claim per row.

One fact per row. Two rows on one fact are a duplicate (Moderate). Three or more fail.

Every row: criterion, justification, evidence, category and weight. The evidence cites the record, matches its value, and the prompt gives a reason to open it.

Weight	Difficulty axes in the row
1	None or one: a value stated in one place
3	Exactly two: compare two sources, surface a mismatch
5	Three or more: reconcile three sources and apply a rule

Essential and easy is still a 1. Emailing the right person may be the point of the task, but the address is in Contacts: weight 1.

Weak criteria, and what not to grade

Defect	Weak criterion	Fix
Not self-contained	The Agent reports the correct refund total.	The Agent states that card refunds for Feb 25-27 total $4,120.
Two jobs on one row	The Agent posts the variance to Slack and creates the Linear ticket.	Two rows: the Slack post, then the Linear ticket, each with the variance.
Title lacks the real value	The Agent includes the 15 taxonomy events.	The Agent includes begin_level, left_level and notification_permission.
Negative framing (recovery)	The Agent does not report the data as unavailable.	The Agent gets the rollback date from the drive copy after the mail search fails.
Passive, or a tool name	A ticket is raised via linear_create_issue.	The Agent raises a ticket for the iOS paywall.
Disjunctive	The Agent files the record in a doc or a Trello list.	The Agent creates a durable record alongside the launch material.
Unstated expectation	The Agent adds a due date to every ticket.	Drop it: the prompt never asked for due dates.
One reading, when two are fair	The Agent reports the cost as $8,500.	Accept both: the precise $8,500 and the rounded $8,000.
Grading the distractor	The Agent explains why the old note is wrong.	Grade the real value; ignoring the distractor is valid.
Overly specific	The Agent creates an issue titled exactly "iOS paywall regression".	The Agent raises a ticket stating that the iOS build still shows level 3.

The last four rows are the ways a rubric gets too strict.

Check the whole set

COVERAGE

Every request needs an Outcome row; a missing one is Major. Repetitive items: spot-check five or fewer, hardest first, plus a volume row. Never a core requirement.

PROCESS

Only for ordering, and only when all three conditions hold. Two needless Process rows fail.

WEIGHTS

Count the axes: sources, reasoning, modality, discovery. None or one is 1, two is 3, three or more is 5.

The rubric fails with more than	Share of rows
Major issues	10%
Moderate or worse	15%
Minor or worse	20%

FAILS ON ITS OWN

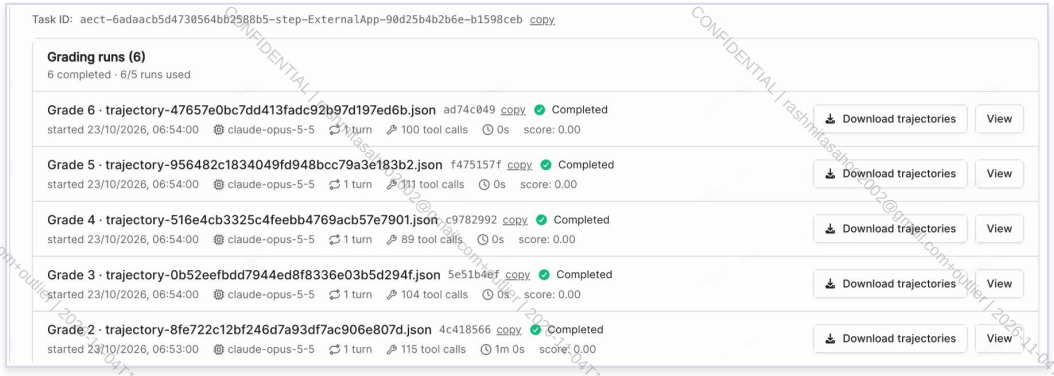
No Outcome rows. More than half Process, or two needless Process rows. One negative criterion. The same fact in three or more rows. Two invalid all-fail rows: every completed run fails them because the row is wrong, not the agent.

Never delete rows to pass difficulty: that creates a missing-coverage defect and leaves the task exactly as easy.

Run the verifier, then read the matrix



Grading results: one column per criterion



1 — All six runs finished

Check the Agent Runner before you grade

2 — Run rubric verifiers once

Each run is graded against your rubric

3 — Read the red columns

Red in every run is an all-fail

4 — Justify or fix

Fix any all-fail that's vague, packed or never asked for

The difficulty check, worked through

Per rollout

#	Passed	Total	Weighted Rate	All-Pass?
1	38	44	86.5%	✗
2	41	44	91.2%	✗
3	39	44	89.9%	✗
4	39	44	88.5%	✗
5	38	44	86.5%	✗
6	41	44	91.2%	✗

If a number here disagrees with the matrix view, trust the matrix and report the discrepancy via the issue form.

Per rollout, from the difficulty check

THIS BASELINE

Average score: about 89.0%
pass@1: 0 of 6 runs pass everything (0%)
Medium, at most 85%: fails
Easy, at most 90%: passes

Check	Target
Difficulty bar	Average score within your mode's ceiling
pass@1 (pass@k)	At most 67%: no more than 4 of 6 runs pass everything
Failure patterns	At least 2; no pattern covers more than half
Tool calls	Above your minimum: on a 100% run if one exists, else the six-run average
Completed runs	All six

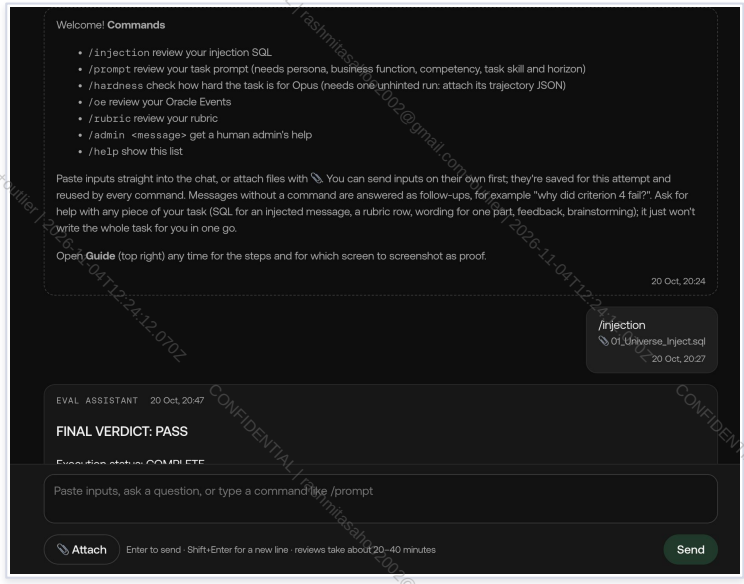
Hard only after fair: fix every unfair row and re-grade before you judge difficulty. Never delete rubrics to move the numbers.

Every review, the same loop

Command	Runs after	What it needs
/injection	Injection SQL	Your SQL
/prompt	The prompt	Prompt and task details
/hardness	Opus pre-check	The pre-check trajectory JSON
/oe	Oracle Events	Oracle Events, plus the prompt, details and SQL
/rubric	The rubric	Your rubric, as JSON

IN THE TASK, AFTER EACH REVIEW

Enter the verdict, its score (1 for PASS or NON-FAIL, 0 for FAIL) and the verification code from the pass screen. FAIL blocks the task.



The web eval: attach the inputs, type the command, send

At least NON-FAIL on every review. A FAIL blocks the task until you fix the artifact and rerun. No proof upload needed.

How a task is scored

Dimension	1 to 2: FAIL	3 to 4: PASS	5
Unique Ground Truth	The rubric accepts only one of two fair readings	Doesn't exist	Every expert agrees
Agent-Centric Phrasing	Passive, or names a tool	Valid but off-pattern	Clean throughout
Negative Criteria	A criterion says what the agent must not do	–	Every criterion says what the agent does
Prompt Clarity	Vague, or two write paths	Wording varies, action doesn't	Unambiguous
Truthfulness	One major or two minor errors	One minor error	All verified

PROMOTION

Two consecutive tasks scoring 4 or above that pass through to delivery.

A SCORE OF 2

Fixable errors and visible potential. Not a removal.

DISABLED

Two consecutive scores of 2 or below, or suspicious behaviour or spam.

The lowest dimension is the score. A strong score elsewhere doesn't make up for a failing one.

Before you submit

- [ ] Your scenario fits every assignment axis, with your assigned persona and universe date
- [ ] Your SQL ran in Database, sits in the Injection SQL field, and /injection reached at least NON-FAIL
- [ ] The prompt is your own words: a plain request, no tool names, nothing pre-solved, one outcome
- [ ] Every write has an Oracle Event and an Outcome criterion; no fact repeated across rows
- [ ] /prompt, /hardness, /oe and /rubric reached at least NON-FAIL, with each verdict and verification code entered
- [ ] The difficulty check passes for your mode, and every all-fail is valid

Stuck? Join the War Room (daily, 12:00 AM to 5:00 PM PT), or type /admin in the web eval to reach a person.

THANK YOU

Questions?

Ask about anything from today: your assignment, the injection, the prompt, the Oracle Events or the rubric.

Every task is different. Verify every name, ID, date and figure in your own universe.

Seed the symptom · write it yourself · one outcome · Outcome first

War Room

QMs and CSMs while you task, daily 12:00 AM to 5:00 PM PT. Your first task is live-reviewed by a QM.

THE GUIDELINE SET

- Overall Task Flow
- Universe and Tools
- Persona ACL Policy
- Prompt Writing
- Oracle Events
- Rubric Writing
- Eval Portal and L0 Review
- Hardness Guide
- Injection Schema
- Common Errors and Pre-QC

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-11-04T12:24:12.070Z